

International Islamic University Chittagong
Department of Computer Science and Engineering



بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Lab Report:
JAVA PRACTICE PROBLEM

Course Code: CSE-2340

Course Title: SOFTWARE DEVELOPMENT 1

Submitted To :
Md. Sadman Hafiz
Lecturer, Dept. of CSE, IIUC

Submitted By :

Mahima Farin
ID: C251212
Semester: 3rd
Section: 3AF
Department : CSE

Signature:

Date of Submission : 08/04/2026

SET 1 (Beginner)

Problem-01: Print “Hello, World!”

Input-Output Screenshot:



```
1
2 package javahello12;
3
4 public class JavaHello12 {
5
6     public static void main(String[] args) {
7
8         System.out.println("Hello, World!");
9     }
10 }
11
```

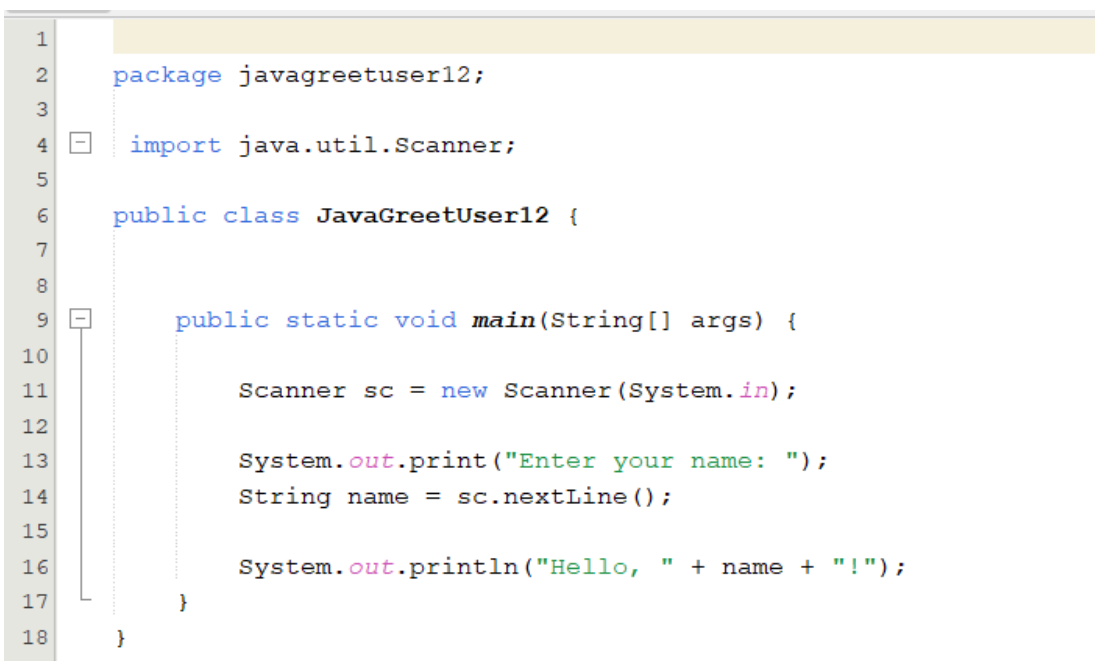
run:
Hello, World!
BUILD SUCCESSFUL (total time: 0 seconds)

Approach:

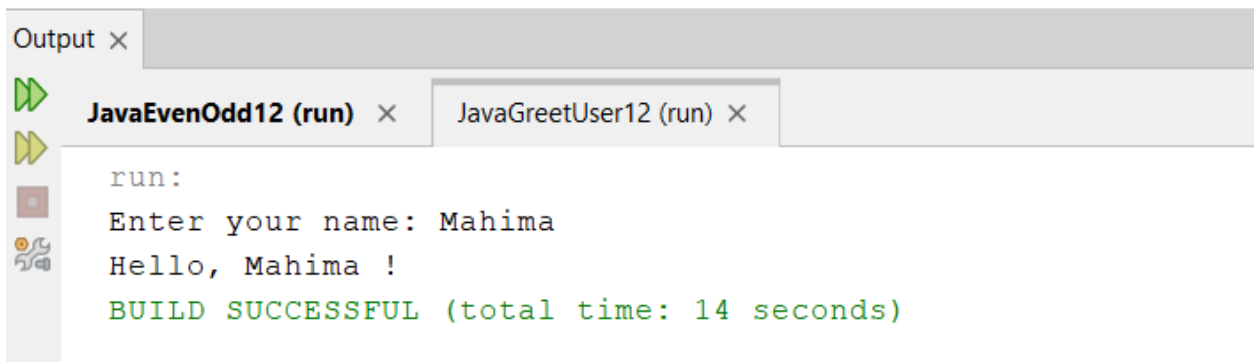
Here, I wrote a simple Java program to display a message. I used the `System.out.println()` method to print "Hello, World!". This program does not require any input.

Problem 2 : Take the user's name and greet them. Example, Input: “Your Name” and Output: Hello, “Your Name”!

Input-Output Screenshot:



```
1
2 package javagreetuser12;
3
4 import java.util.Scanner;
5
6 public class JavaGreetUser12 {
7
8
9     public static void main(String[] args) {
10
11         Scanner sc = new Scanner(System.in);
12
13         System.out.print("Enter your name: ");
14         String name = sc.nextLine();
15
16         System.out.println("Hello, " + name + "!");
17     }
18 }
```



```
Output x
JavaEvenOdd12 (run) x  JavaGreetUser12 (run) x
run:
Enter your name: Mahima
Hello, Mahima !
BUILD SUCCESSFUL (total time: 14 seconds)
```

Approach:

Here, I used the Scanner class to take the user's name as input. The input is stored in a string variable.

Then, I printed a greeting message by concatenating "Hello" with the user's name

Problem 3: Check if a number is even or odd

Input-Output Screenshot:



```
1 package javaevenodd12;
2
3
4 import java.util.Scanner;
5
6 public class JavaEvenOdd12 {
7
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         System.out.print("Enter a number: ");
13         int num = sc.nextInt();
14
15         if (num % 2 == 0) {
16             System.out.println("Even");
17         } else {
18             System.out.println("Odd");
19         }
20     }
21 }
22
```

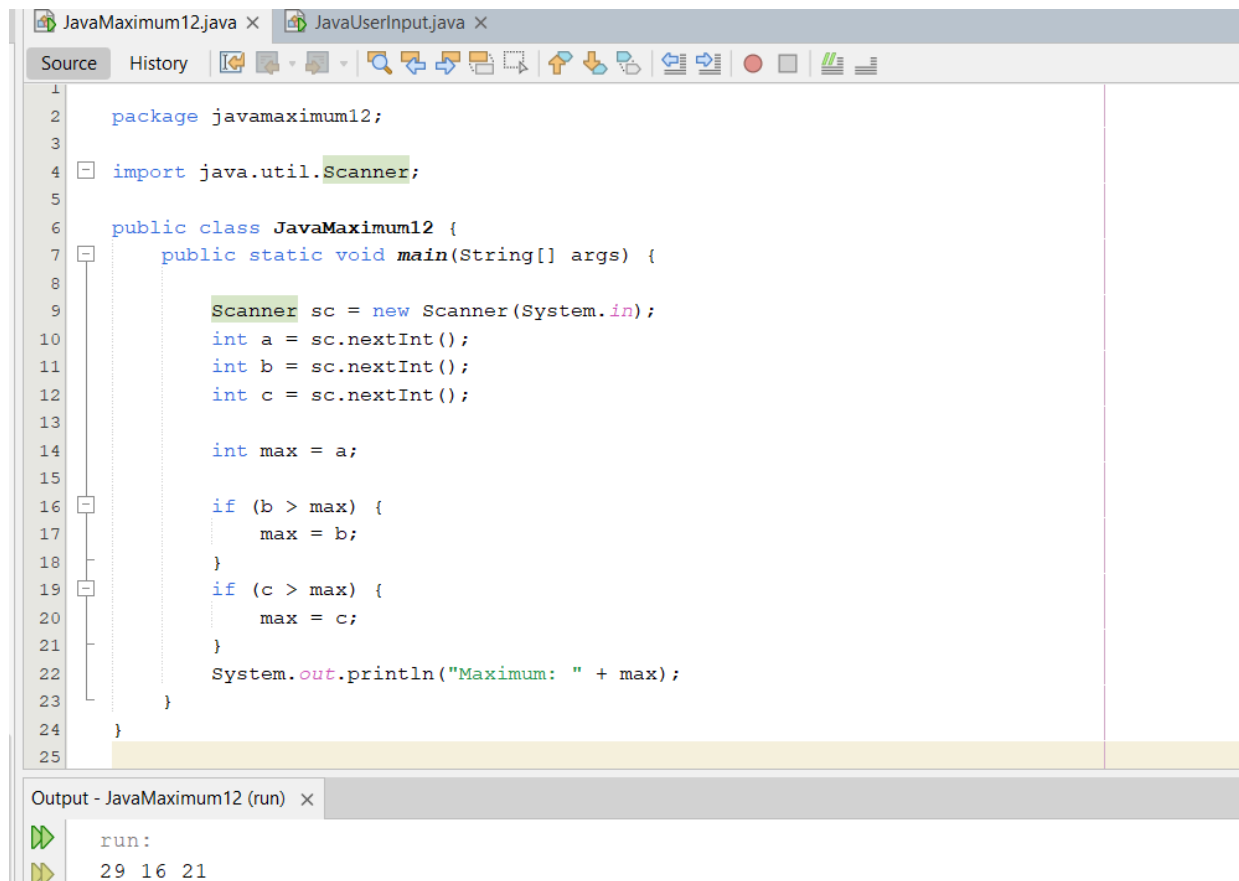
```
Output - JavaEvenOdd12 (run) x
run:
Enter a number: 22
Even
BUILD SUCCESSFUL (total time: 9 seconds)
```

Approach:

Here, I took an integer input from the user using Scanner. Then, I used the modulus operator % to divide the number by 2. If the remainder is 0, the number is even; otherwise, it is odd.

Problem 4: Find the maximum of 3 numbers: Take 3 numbers from user input, print the largest.

Input-Output Screenshot:



The screenshot shows an IDE with two tabs: `JavaMaximum12.java` and `JavaUserInput.java`. The `Source` tab is active, displaying the following Java code:

```
1 package javamaximum12;
2
3
4 import java.util.Scanner;
5
6 public class JavaMaximum12 {
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10        int a = sc.nextInt();
11        int b = sc.nextInt();
12        int c = sc.nextInt();
13
14        int max = a;
15
16        if (b > max) {
17            max = b;
18        }
19        if (c > max) {
20            max = c;
21        }
22        System.out.println("Maximum: " + max);
23    }
24 }
25
```

Below the code editor, the `Output - JavaMaximum12 (run)` window shows the execution results:

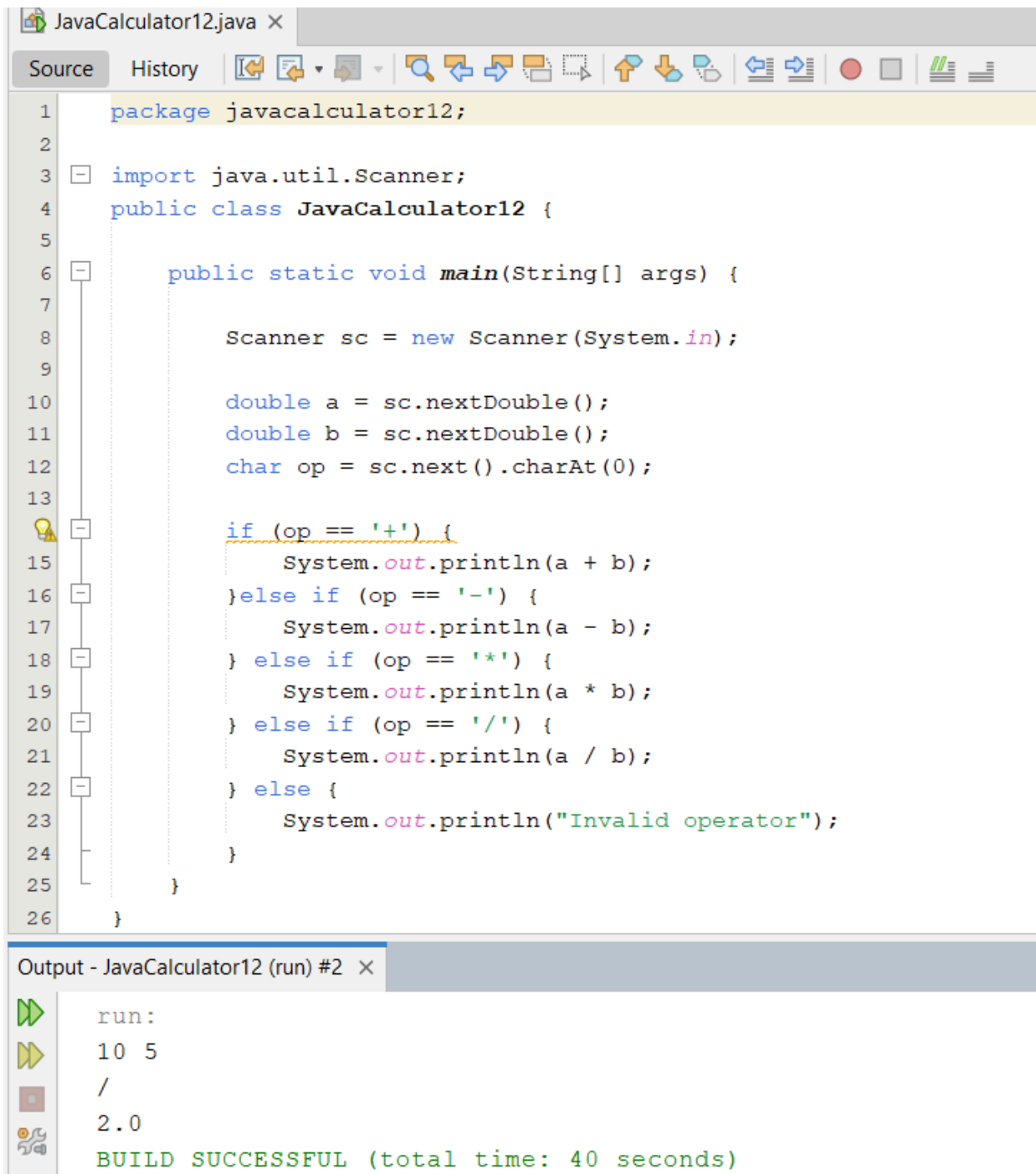
```
run:
29 16 21
```

Approach:

Here, I took three numbers as input from the user. I assumed the first number as the maximum. Then, I compared it with the other numbers and updated the maximum accordingly.

Problem 5: Simple Calculator (Add, Subtract, Multiply, Divide): Use if-else or switch to perform operations.

Input-Output Screenshot:



The screenshot displays an IDE window titled "JavaCalculator12.java". The code defines a package, imports the Scanner class, and creates a JavaCalculator12 class with a main method. The main method uses Scanner to read two double values and a character operator. It then uses an if-else statement to perform the appropriate arithmetic operation and print the result. The output window shows the execution of the program with inputs 10, 5, and /, resulting in 2.0. The build was successful.

```
1 package javacalculator12;
2
3 import java.util.Scanner;
4 public class JavaCalculator12 {
5
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9
10        double a = sc.nextDouble();
11        double b = sc.nextDouble();
12        char op = sc.next().charAt(0);
13
14        if (op == '+') {
15            System.out.println(a + b);
16        } else if (op == '-') {
17            System.out.println(a - b);
18        } else if (op == '*') {
19            System.out.println(a * b);
20        } else if (op == '/') {
21            System.out.println(a / b);
22        } else {
23            System.out.println("Invalid operator");
24        }
25    }
26 }
```

Output - JavaCalculator12 (run) #2 ×

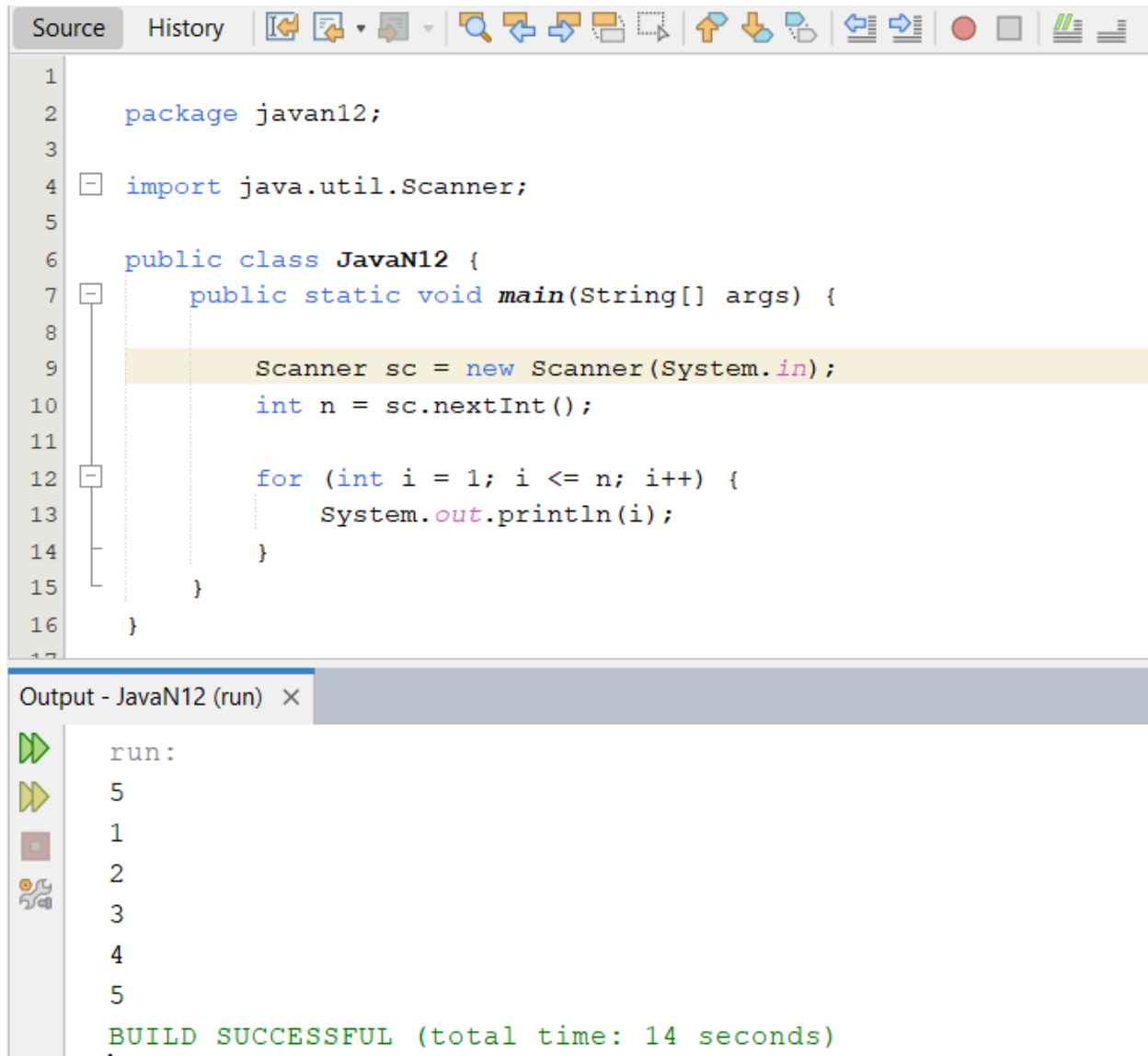
```
run:
10 5
/
2.0
BUILD SUCCESSFUL (total time: 40 seconds)
```

Approach:

Here, I took two numbers and an operator as input. Then, I used if-else statements to perform the required operation. The result is displayed based on the chosen operator.

Problem 6: Print numbers from 1 to N. Take N from the user input.

Input-Output Screenshot:



The screenshot displays an IDE with a Java source file and its execution output. The source code defines a package, imports the Scanner class, and implements a main method that reads an integer N and prints numbers from 1 to N using a for loop. The output window shows the input '5' and the resulting sequence of numbers '1' through '5', followed by a success message.

```
1
2 package javan12;
3
4 import java.util.Scanner;
5
6 public class JavaN12 {
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10        int n = sc.nextInt();
11
12        for (int i = 1; i <= n; i++) {
13            System.out.println(i);
14        }
15    }
16 }
```

Output - JavaN12 (run) ×

```
run:
5
1
2
3
4
5
BUILD SUCCESSFUL (total time: 14 seconds)
```

Approach:

I took a number **N** as input from the user. Then, I used a **for loop** that starts from 1 and continues up to **N**. In each iteration of the loop, the current number is printed. As a result, all numbers from 1 to **N** are displayed sequentially

Problem 7: Print the multiplication table of a number

Input-Output Screenshot:



```
1
2 package javatable12;
3
4 import java.util.Scanner;
5 public class JavaTable12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        int num = sc.nextInt();
11
12        for (int i = 1; i <= 10; i++) {
13            System.out.println(num + " x " + i + " = " + (num * i));
14        }
15    }
16 }
```

Output - JavaTable12 (run) ×

run:
3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

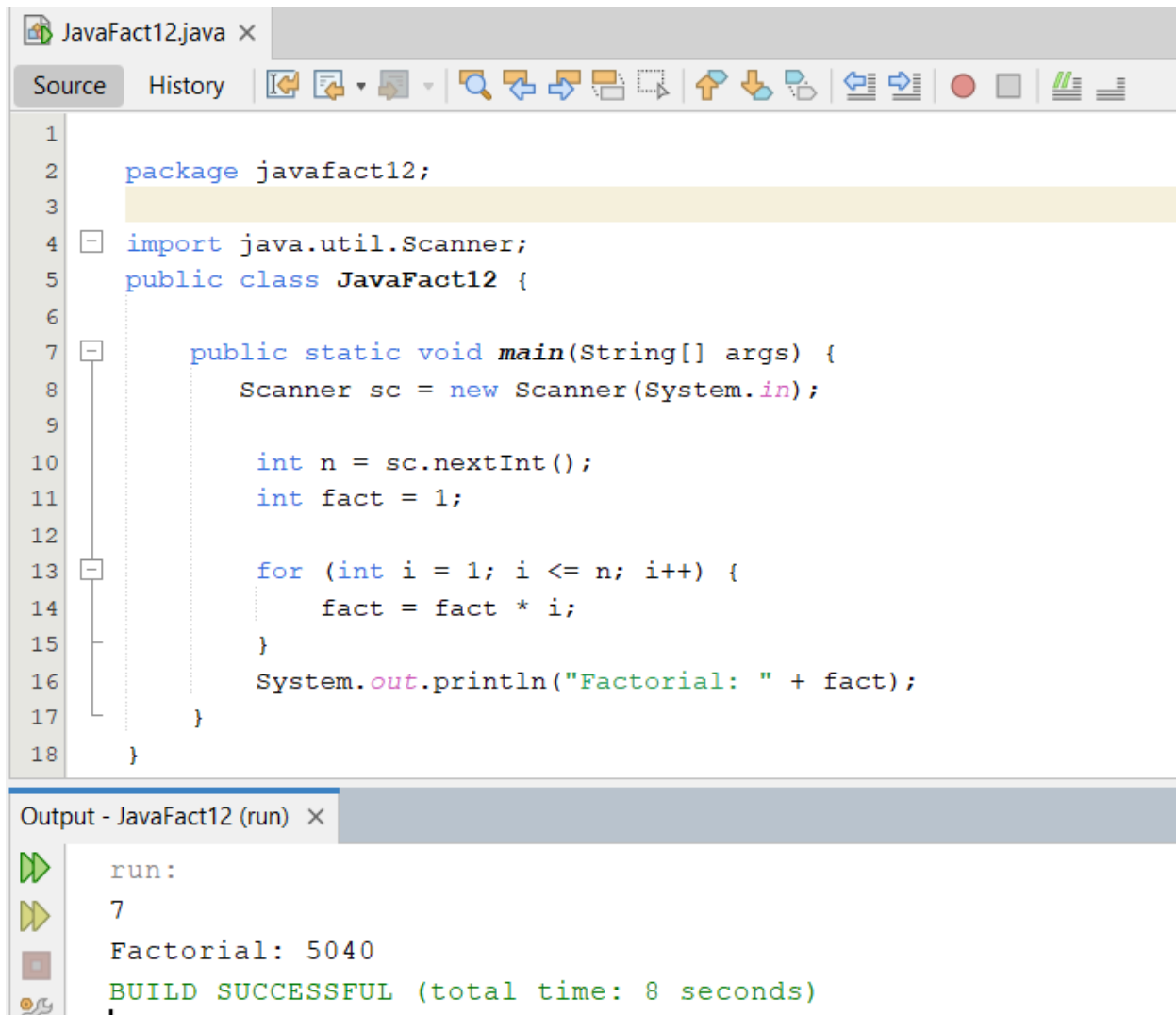
Approach:

Here, I took a number as input. Then, I used a loop from 1 to 10. In each iteration, I multiplied the number and displayed the result.

Problem 8:

Calculate the factorial of a number. Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Input-Output Screenshot:



The screenshot displays an IDE window titled 'JavaFact12.java'. The code defines a package 'javafact12', imports 'java.util.Scanner', and creates a public class 'JavaFact12'. Inside the class, the 'main' method takes a 'String[] args' array, creates a 'Scanner' object 'sc' from 'System.in', reads an integer 'n', initializes 'fact' to 1, and uses a 'for' loop to calculate the factorial of 'n'. The result is printed as 'Factorial: ' followed by 'fact'.

```
1
2 package javafact12;
3
4 import java.util.Scanner;
5 public class JavaFact12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        int n = sc.nextInt();
11        int fact = 1;
12
13        for (int i = 1; i <= n; i++) {
14            fact = fact * i;
15        }
16        System.out.println("Factorial: " + fact);
17    }
18 }
```

Below the code editor is the 'Output - JavaFact12 (run)' window. It shows the execution of the program with the input '7' and the output 'Factorial: 5040'. It also indicates a successful build with a total time of 8 seconds.

```
run:
7
Factorial: 5040
BUILD SUCCESSFUL (total time: 8 seconds)
```

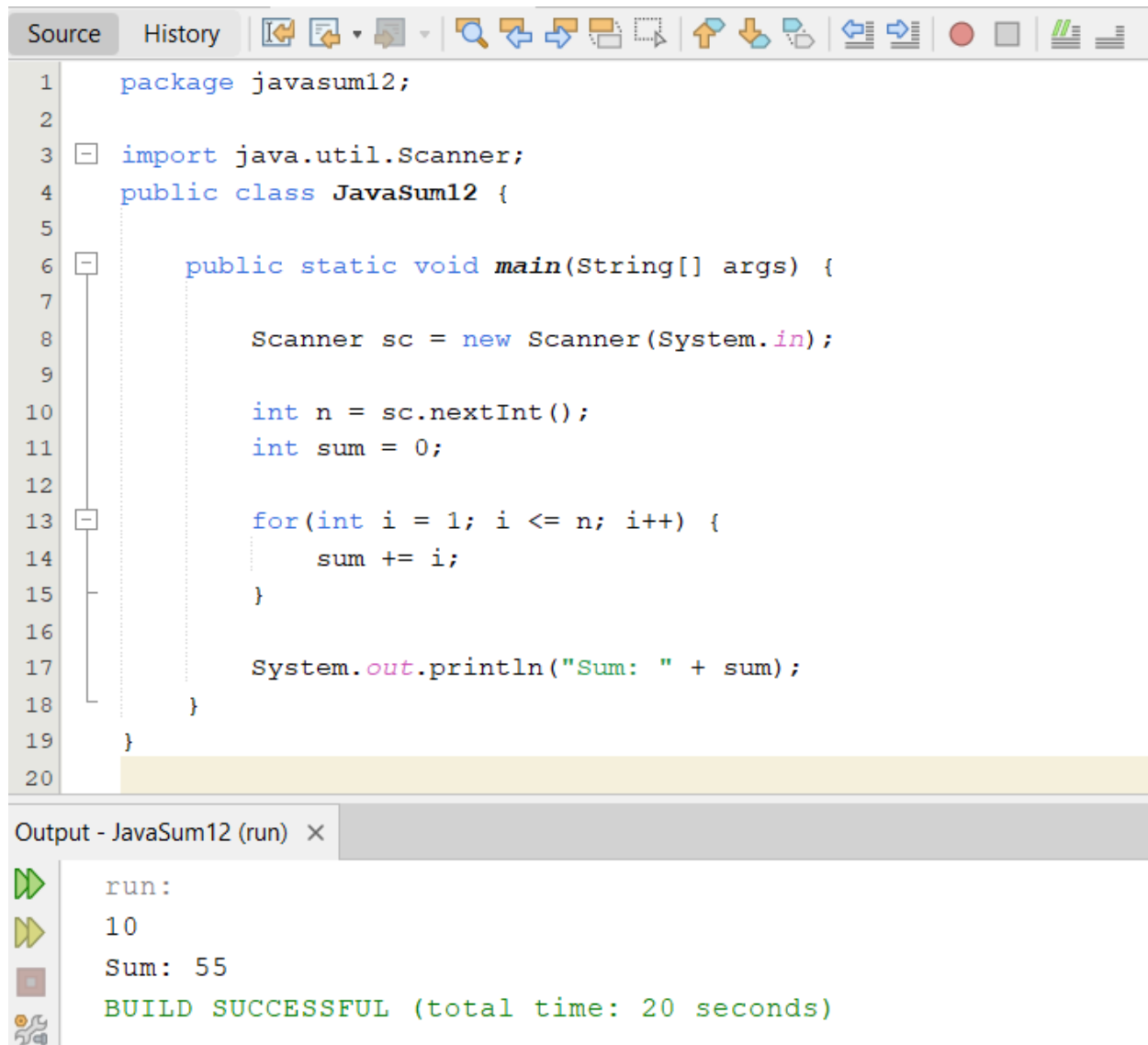
Approach:

Here, I took a number as input from the user. I initialized a variable to store the factorial value. Then, I used a loop to multiply numbers from 1 to N.

Problem 9:

Sum of all numbers from 1 to N

Input-Output Screenshot:



The screenshot displays an IDE window with a Java source file named `JavaSum12.java`. The code implements a program to calculate the sum of integers from 1 to a given number `n`. The code is as follows:

```
1 package javasum12;
2
3 import java.util.Scanner;
4 public class JavaSum12 {
5
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9
10        int n = sc.nextInt();
11        int sum = 0;
12
13        for(int i = 1; i <= n; i++) {
14            sum += i;
15        }
16
17        System.out.println("Sum: " + sum);
18    }
19 }
20
```

Below the source editor, the 'Output' window shows the execution results for `JavaSum12 (run)`:

```
run:
10
Sum: 55
BUILD SUCCESSFUL (total time: 20 seconds)
```

Approach:

Here, I took a number `N` as input. I initialized a variable to store the sum. Then, I used a loop to add numbers from 1 to `N` and printed the result.

SET 2 (Intermediate)

Problem 1: Store and print 5 numbers in an array

Input-Output Screenshot:

```
1
2 package javaarrayprint12;
3 import java.util.Scanner;
4
5 public class JavaArrayPrint12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int arr[] = new int[5];
10
11         for (int i = 0; i < 5; i++) {
12             arr[i] = sc.nextInt();
13         }
14
15         for (int i = 0; i < 5; i++) {
16             System.out.println(arr[i]);
17         }
18     }
19
20 }
```

Output - JavaArrayPrint12 (run) ×

run:
30 5 36 76 12 343
30
5
36
76
12
BUILD SUCCESSFUL (total time: 15 seconds)

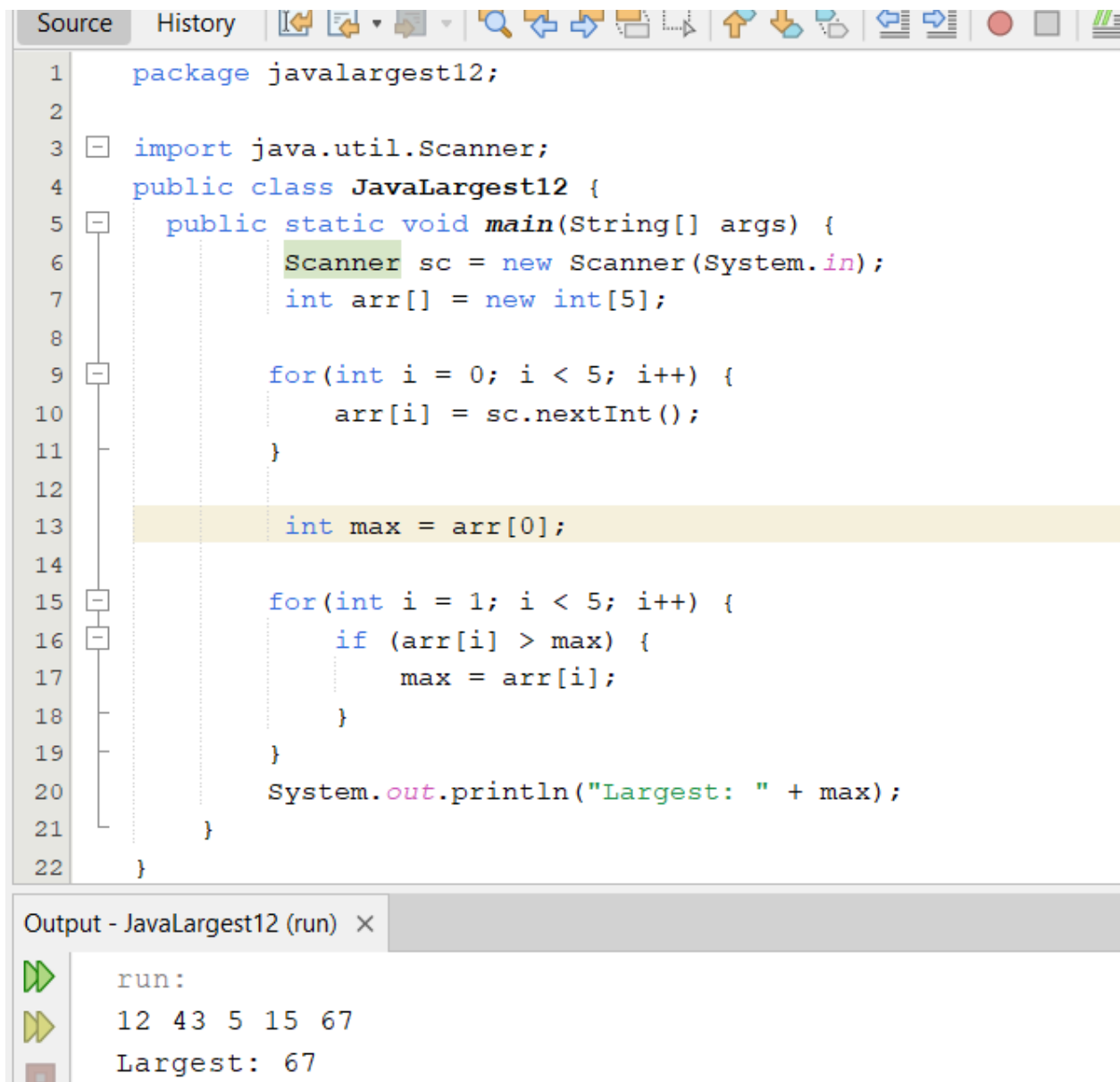
Approach:

In this program, I created an integer array of size 5 to store the numbers entered by the user. Using the Scanner class, I took input from the user in a for loop and stored each number into the array. After all numbers were stored, I used another for loop to access each element of the array and

printed them one by one using `System.out.println()`. This approach demonstrates basic array storage, input handling, and iteration using loops

Problem 2: Find the largest number in an array

Input-Output Screenshot:



```
1 package javalargest12;
2
3 import java.util.Scanner;
4 public class JavaLargest12 {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         int arr[] = new int[5];
8
9         for(int i = 0; i < 5; i++) {
10             arr[i] = sc.nextInt();
11         }
12
13         int max = arr[0];
14
15         for(int i = 1; i < 5; i++) {
16             if (arr[i] > max) {
17                 max = arr[i];
18             }
19         }
20         System.out.println("Largest: " + max);
21     }
22 }
```

Output - JavaLargest12 (run) ×

```
run:
12 43 5 15 67
Largest: 67
```

Approach:

Here, I stored numbers in an integer array and assumed the first element as the maximum. Using a loop, I compared this value with each of the other elements in the array. If a larger number was found, I updated the maximum value. After completing the loop, the largest number was printed. This demonstrates the use of arrays, loops, and conditional statements to find the maximum value among a set of numbers.

Problem 3: Find the smallest number in an array

Input-Output Screenshot:

```
1  package javasmallest12;
2  import java.util.Scanner;
3
4  public class JavaSmallest12 {
5
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          int arr[] = new int[5];
9
10         for(int i = 0; i < 5; i++) {
11             arr[i] = sc.nextInt();
12         }
13         int min = arr[0];
14
15         for(int i = 1; i < 5; i++) {
16             if (arr[i] < min) {
17                 min = arr[i];
18             }
19         }
20         System.out.println("Smallest: " + min);
21     } }
22
```

Output - JavaSmallest12 (run) ×

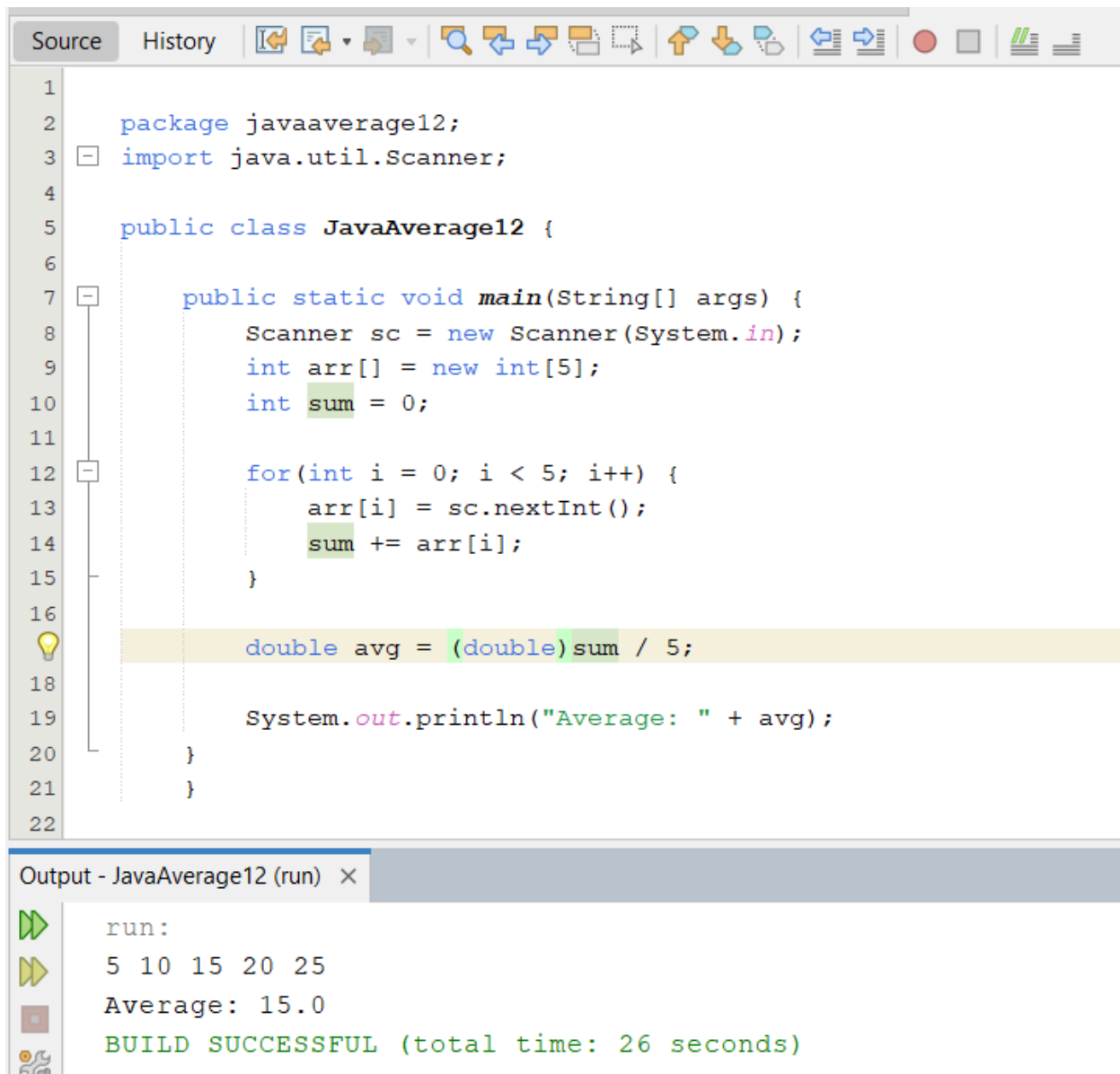
```
run:
1 43 54 24 34 56 7
Smallest: 1
BUILD SUCCESSFUL (total time: 23 seconds)
```

Approach:

In this program, I stored numbers in an array and assumed the first element as the minimum. I then compared each subsequent element with this value using a loop, updating the minimum whenever a smaller number was found. After traversing the entire array, I printed the smallest number. This approach helps in understanding arrays, iteration, and conditional checking to determine the minimum value.

Problem 4: Calculate the average of an array

Input-Output Screenshot:



The screenshot displays an IDE with a Java source file and its execution output. The source code defines a class `JavaAverage12` with a `main` method. It uses a `Scanner` to read five integers into an array, calculates their sum, and then computes the average by casting the sum to a `double` and dividing by 5. The output window shows the program running successfully, displaying the input numbers, the calculated average, and a build status message.

```
1
2 package javaaverage12;
3 import java.util.Scanner;
4
5 public class JavaAverage12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int arr[] = new int[5];
10        int sum = 0;
11
12        for(int i = 0; i < 5; i++) {
13            arr[i] = sc.nextInt();
14            sum += arr[i];
15        }
16
17        double avg = (double)sum / 5;
18
19        System.out.println("Average: " + avg);
20    }
21 }
22
```

Output - JavaAverage12 (run) ×

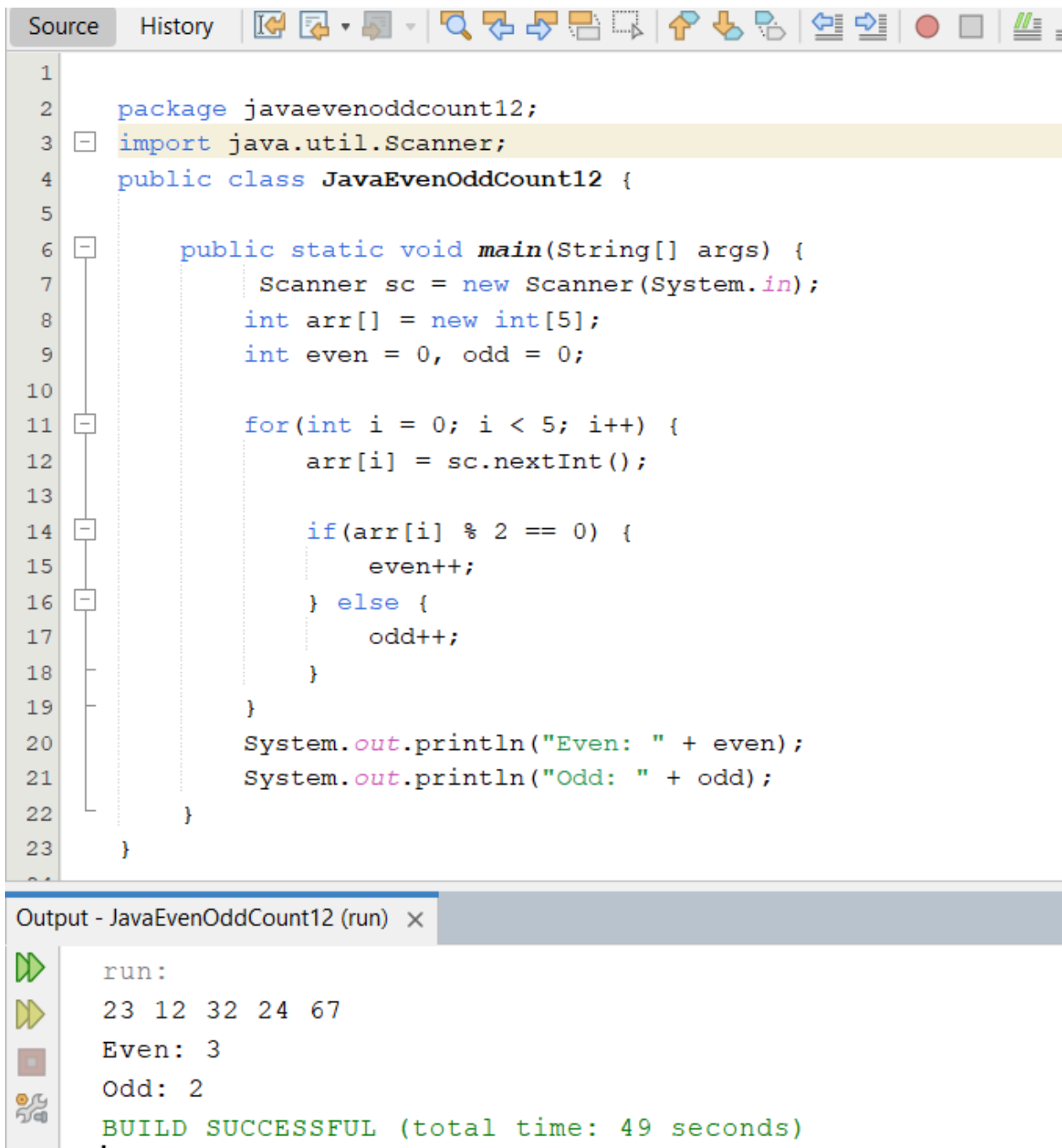
```
run:
5 10 15 20 25
Average: 15.0
BUILD SUCCESSFUL (total time: 26 seconds)
```

Approach:

I stored numbers in an array of size 5 and initialized a variable sum to 0. Using a loop, I added each element of the array to calculate the total sum. Then, I divided the sum by the number of elements to find the average and stored it in a double variable. Finally, the average value was printed.

Problem 5: Count even and odd numbers in an array

Input-Output Screenshot:



The screenshot displays an IDE with a Java source file and its execution output. The source code, located in the upper pane, defines a package `javaevenoddcount12`, imports `java.util.Scanner`, and creates a class `JavaEvenOddCount12`. The `main` method initializes a `Scanner` object, an array of 5 integers, and two counters, `even` and `odd`, both set to 0. A `for` loop iterates over the array, reading each element. For each element, it checks if it is even using the modulus operator `% 2 == 0`. If true, the `even` counter is incremented; otherwise, the `odd` counter is incremented. After the loop, the counts are printed using `System.out.println`. The lower pane shows the output of the program, which includes the input numbers, the counts, and a successful build message.

```
1
2 package javaevenoddcount12;
3 import java.util.Scanner;
4 public class JavaEvenOddCount12 {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int arr[] = new int[5];
9         int even = 0, odd = 0;
10
11         for(int i = 0; i < 5; i++) {
12             arr[i] = sc.nextInt();
13
14             if(arr[i] % 2 == 0) {
15                 even++;
16             } else {
17                 odd++;
18             }
19         }
20         System.out.println("Even: " + even);
21         System.out.println("Odd: " + odd);
22     }
23 }
```

Output - JavaEvenOddCount12 (run) x

```
run:
23 12 32 24 67
Even: 3
Odd: 2
BUILD SUCCESSFUL (total time: 49 seconds)
```

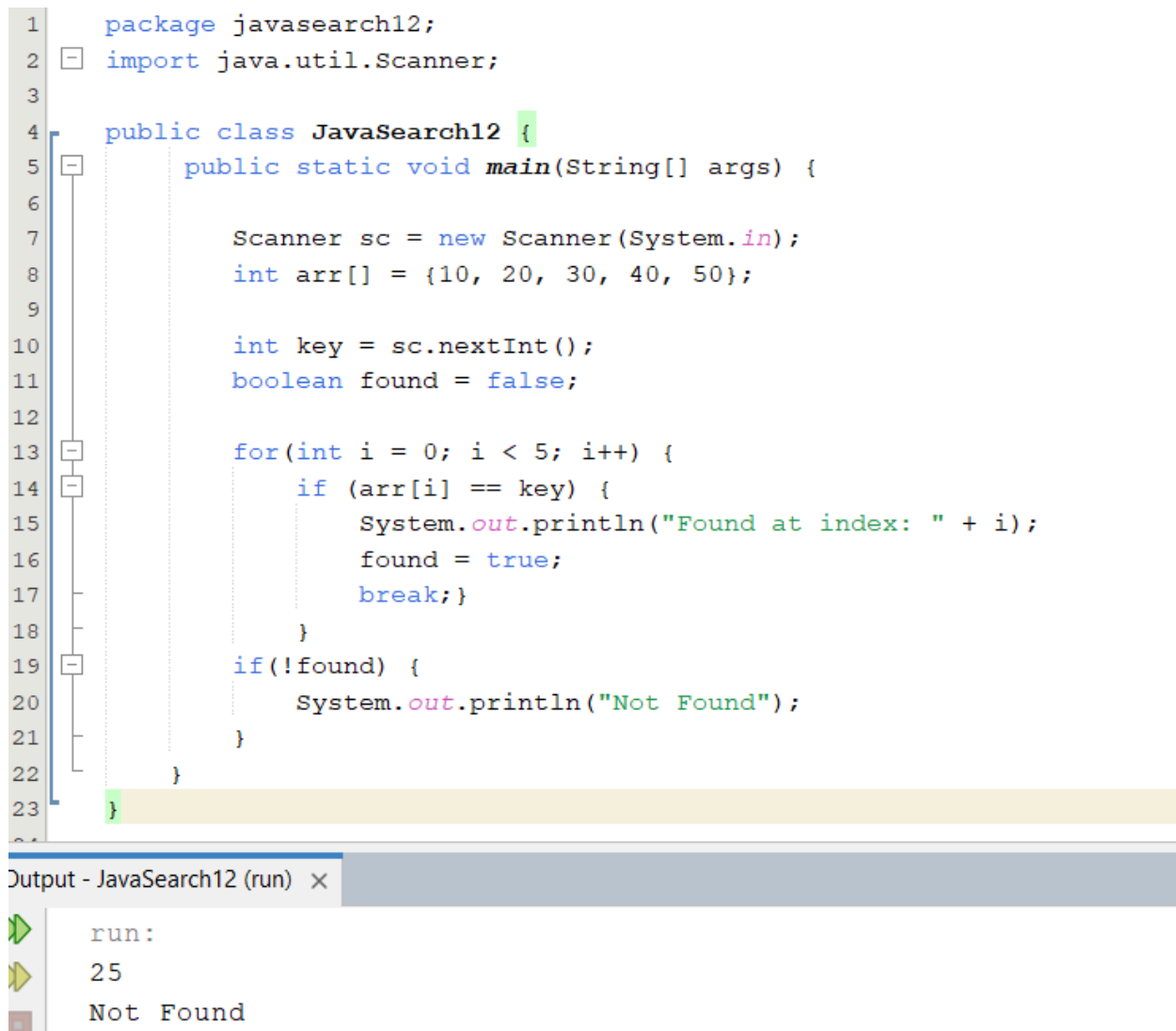
Approach:

In this program, I stored numbers in an array and initialized two counters, even and odd, to zero. Using a loop, I checked each element with the modulus operator `% 2`. If a number was divisible by 2, the even counter was incremented; otherwise, the odd counter was incremented. Finally, both counts were printed.

Problem 6:

Search for an element in an array

Input-Output Screenshot:



```
1 package javasearch12;
2 import java.util.Scanner;
3
4 public class JavaSearch12 {
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8         int arr[] = {10, 20, 30, 40, 50};
9
10        int key = sc.nextInt();
11        boolean found = false;
12
13        for(int i = 0; i < 5; i++) {
14            if (arr[i] == key) {
15                System.out.println("Found at index: " + i);
16                found = true;
17                break;}
18        }
19        if(!found) {
20            System.out.println("Not Found");
21        }
22    }
23 }
```

Output - JavaSearch12 (run) x

run:
25
Not Found

Approach:

Here, I stored elements in an array and took a number as input from the user to search. Using a loop, I compared each element with the input value. If a match was found, I printed the index and set a boolean variable found to true. If no match was found after the loop, I displayed "Not Found".

Problem 7:

Sort an array (Ascending Order)

Input-Output Screenshot:

```
2 package javasortarray12;
3
4 public class JavaSortArray12 {
5     public static void main(String[] args) {
6
7         int arr[] = {5, 3, 1, 4, 2};
8
9         for(int i = 0; i < arr.length; i++) {
10             for (int j = i + 1; j < arr.length; j++) {
11                 if (arr[i] > arr[j]) {
12                     int temp = arr[i];
13                     arr[i] = arr[j];
14                     arr[j] = temp;
15                 }
16             }
17         }
18         for(int i = 0; i < arr.length; i++) {
19             System.out.println(arr[i]);
20         }
21     }
22 }
```

Output - JavaSortArray12 (run) ×

run:
1
2
3
4
5
BUILD SUCCESSFUL (total time: 0 seconds)

Approach:

In this program, I stored numbers in an array and used nested loops to compare elements. Whenever an element was greater than the following element, I swapped their positions using a temporary variable. After completing the loops, the array was arranged in ascending order, and I printed all the elements. This demonstrates array sorting using comparison, swapping, and nested loops.

Problem 8:

Check if a string is a palindrome

Input-Output Screenshot:

```
2 package javapalindrome12;
3 import java.util.Scanner;
4
5 public class JavaPalindrome12 {
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9         String str = sc.nextLine();
10
11         String rev = "";
12         for (int i = str.length() - 1; i >= 0; i--) {
13             rev = rev + str.charAt(i);
14         }
15
16         if (str.equals(rev)) {
17             System.out.println("Palindrome");
18         } else {
19             System.out.println("Not Palindrome");
20         }
21     }
22 }
```

Output - JavaPalindrome12 (run) ×

run:
mom
Palindrome
BUILD SUCCESSFUL (total time: 26 seconds)

Approach:

Here, I took a string input from the user and initialized an empty string to store the reversed version. Using a loop, I traversed the original string from the last character to the first, building the reversed string. After reversing, I compared it with the original string using the `.equals()` method. If they were identical, I printed "Palindrome"; otherwise, "Not Palindrome".

Problem 9:

Count vowels and consonants in a string

Input-Output Screenshot:

```
1 package javavowelconsonant12;
2 import java.util.Scanner;
3
4 public class JavaVowelConsonant12 {
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8         String str = sc.nextLine().toLowerCase();
9
10        int v = 0, c = 0;
11        for (int i = 0; i < str.length(); i++) {
12            char ch = str.charAt(i);
13
14            if (ch >= 'a' && ch <= 'z') {
15                if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
16                    v++;
17                } else {
18                    c++;
19                }
20            }
21        }
22        System.out.println("Vowels: " + v);
23        System.out.println("Consonants: " + c);
24    }
25 }
```

Output - JavaVowelConsonant12 (run) ×

```
run:
consonant
Vowels: 3
Consonants: 6
BUILD SUCCESSFUL (total time: 30 seconds)
```

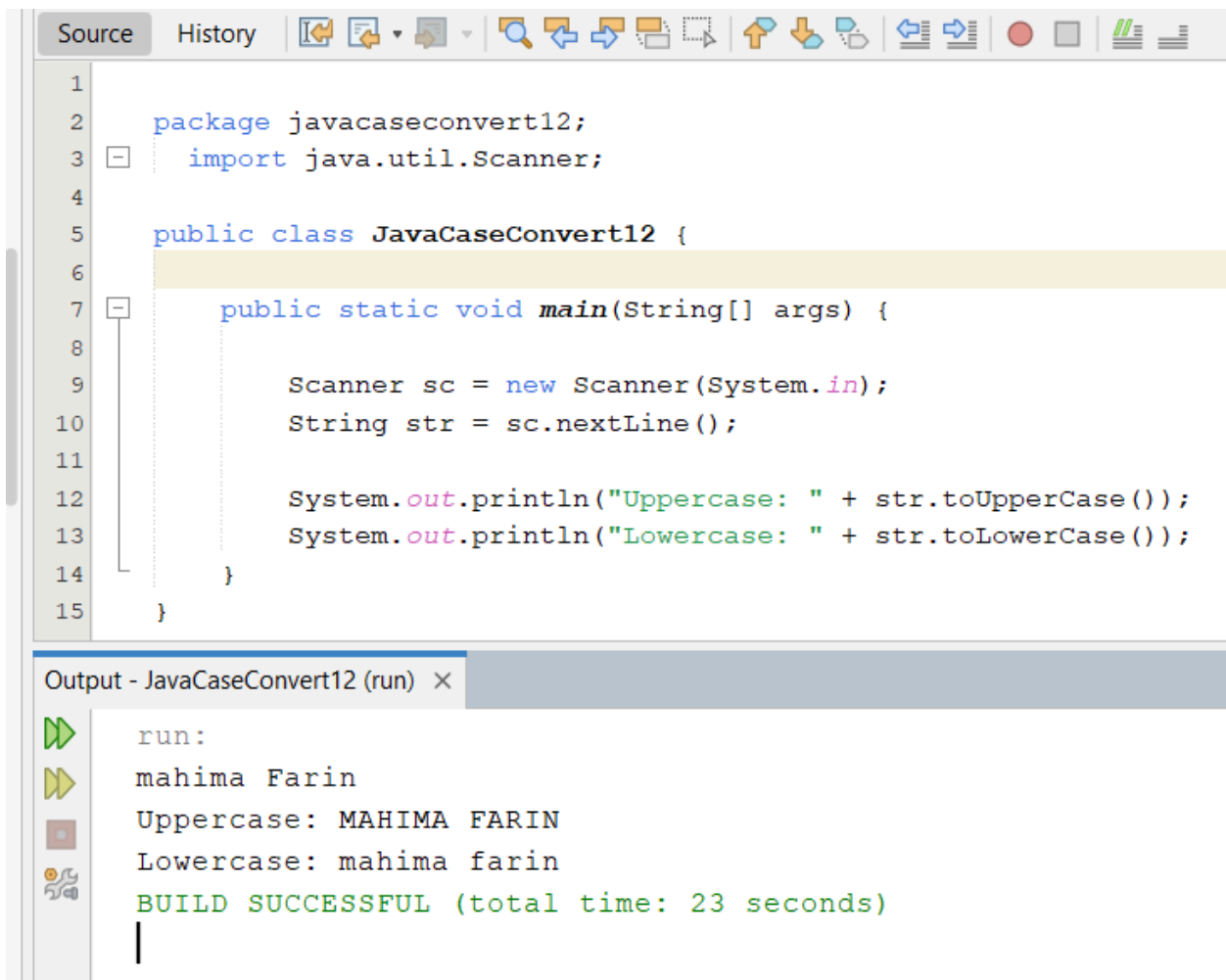
Approach:

In this program, I took a string input and converted it to lowercase to simplify comparisons. I initialized two counters, v for vowels and c for consonants. Using a loop, I checked each character; if it was a letter, I determined whether it was a vowel or consonant and incremented the corresponding counter. After completing the loop, I printed the counts.

Problem 10:

Convert a string to uppercase and lowercase

Input-Output Screenshot:



The screenshot shows an IDE with a source editor and an output console. The source editor contains the following Java code:

```
1
2 package javacaseconvert12;
3 import java.util.Scanner;
4
5 public class JavaCaseConvert12 {
6
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10        String str = sc.nextLine();
11
12        System.out.println("Uppercase: " + str.toUpperCase());
13        System.out.println("Lowercase: " + str.toLowerCase());
14    }
15 }
```

The output console, titled "Output - JavaCaseConvert12 (run)", shows the following output:

```
run:
mahima Farin
Uppercase: MAHIMA FARIN
Lowercase: mahima farin
BUILD SUCCESSFUL (total time: 23 seconds)
```

Approach:

Here, I took a string input from the user. I used the built-in methods `.toUpperCase()` and `.toLowerCase()` to convert the string to uppercase and lowercase, respectively. Both converted strings were printed using `System.out.println()`.

SET 3 (Descriptive Problems)

Problem 1: You are given 2 integers - A and C. You need to find if there exists any integer B that meets the following condition:

1) B must be an integer 2) B is the average of A and C

Input-Output Screenshot:

```
2  package javafindint;
3  import java.util.Scanner;
4
5  public class JavaFindInt {
6      public static void main(String[] args) {
7
8          Scanner sc = new Scanner(System.in);
9          int A = sc.nextInt();
10         int C = sc.nextInt();
11
12         if ((A + C) % 2 == 0) {
13             int B = (A + C) / 2;
14             System.out.println("B = " + B);
15         } else {
16             System.out.println("No integer B exists");
17         }
18     }
19 }
20
```

Output - JavaFindInt (run) ×

run:
4 8
B = 6
BUILD SUCCESSFUL (total time: 5 seconds)

Approach:

In this program I took two integers A and C as input from the user. I checked whether their sum is even using the modulus operator % 2. If the sum is even then an integer B exists and is calculated as $(A + C) / 2$. Otherwise no integer B can exist and I printed the result accordingly.

Problem 2: Let's consider a word too long, if its length is strictly more than 10 characters. All too long words should be replaced with a special abbreviation. This abbreviation is made like this: we write the first and last letters of a word, and between them we write the number of letters between

them. That number is in decimal and doesn't contain any leading zeros. Thus, "localization" will be spelt as "l10n", and "internationalization" will be spelt as "i18n". You are suggested to automate the process of changing words with abbreviations. At that, all too long words should be replaced by the abbreviation and the words that are not too long should not undergo any changes.

Input-Output Screenshot:

```
package javalongword12;
import java.util.Scanner;
public class JavaLongWord12 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String word = sc.nextLine();

        if(word.length() > 10) {
            System.out.println("" + word.charAt(0)+(word.length()-2)+ word.charAt(word.length()-1));
        } else {
            System.out.println(word);
        }
    }
}
```

Output - JavaLongWord12 (run) ×

```
run:
localization
l10n
BUILD SUCCESSFUL (total time: 6 seconds)
```

Approach:

Here I took a word as input from the user and checked its length using `length()`. If the length is greater than 10 then I formed an abbreviation using the first character the number of middle characters (`length - 2`) and the last character. Otherwise I printed the word unchanged.

Problem 3: Capitalization is writing a word with its first letter as a capital letter. Your task is to capitalize the given word.

Input-Output Screenshot:

```
package javacapital12;
import java.util.Scanner;

public class JavaCapital12 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String word = sc.nextLine();

        if(word.length() > 0){
            String result = Character.toUpperCase(word.charAt(0))+ word.substring(1);
            System.out.println(result);
        }
    }
}
```

```
16 }
17
Output - JavaCapital12 (run) x
run:
library
Library
BUILD SUCCESSFUL (total time: 17 seconds)
```

Approach:

In this program I took a string as input from the user. I converted the first character to uppercase using `Character.toUpperCase()` and kept the rest of the string unchanged using `substring()`. Then I printed the final capitalized string.

Problem 4: You are given three digits, a, b, and c. Two of them are equal, but the third one different from the other two. Find the value that occurs exactly once.

Input-Output Screenshot:

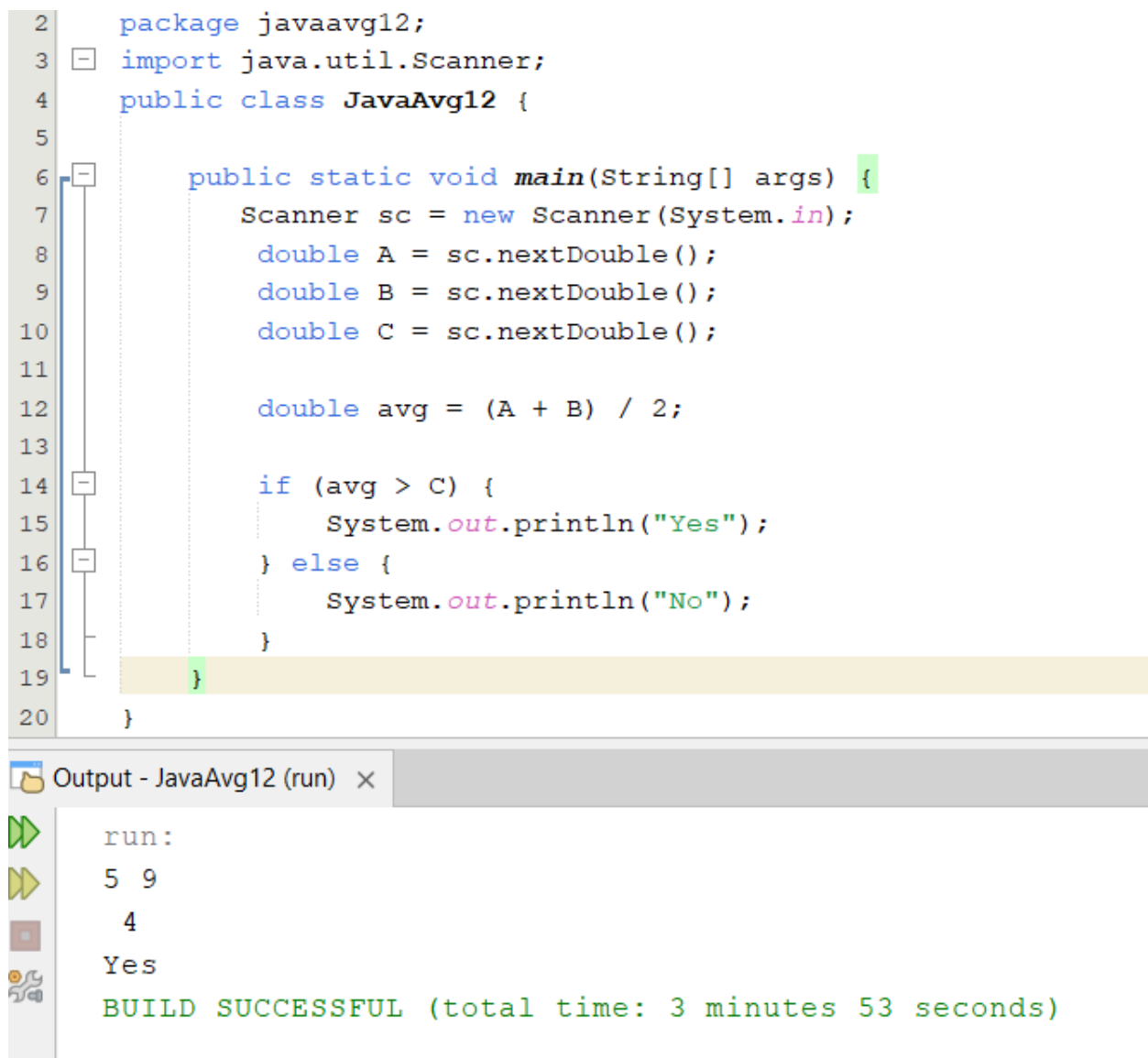
```
1
2 package javaunique12;
3 import java.util.Scanner;
4
5 public class JavaUnique12 {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int a = sc.nextInt();
9         int b = sc.nextInt();
10        int c = sc.nextInt();
11
12        if (a == b) System.out.println(c);
13        else if (a == c) System.out.println(b);
14        else System.out.println(a);
15    }
16 }
17
18
19
Output - JavaUnique12 (run) x
run:
5 3 5
3
BUILD SUCCESSFUL (total time: 1 minute 33 seconds)
```

Approach:

Here I took three integers as input. Since two numbers are equal and one is different I used if-else conditions to compare them. When two values matched I printed the third value which is the unique one.

Problem 5: You are given 3 numbers A, B, and C. Determine whether the average of A and B is strictly greater than C or not? NOTE: Average of A and B is defined as $(A+B)/2$. For example, the average of 5 and 9 is 7, and the average of 5 and 8 is 6.5.

Input-Output Screenshot:



```
2 package javaavg12;
3 import java.util.Scanner;
4 public class JavaAvg12 {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         double A = sc.nextDouble();
9         double B = sc.nextDouble();
10        double C = sc.nextDouble();
11
12        double avg = (A + B) / 2;
13
14        if (avg > C) {
15            System.out.println("Yes");
16        } else {
17            System.out.println("No");
18        }
19    }
20 }
```

Output - JavaAvg12 (run) x

run:
5 9
4
Yes
BUILD SUCCESSFUL (total time: 3 minutes 53 seconds)

Approach:

In this program I took three numbers A B and C as input. I calculated the average of A and B using $(A + B) / 2$. Then I compared the result with C and printed Yes if it is strictly greater otherwise No.

Problem 6: A new TV streaming service, IIUC-TV, was recently launched in IIUC Hall. A group of N friends in IIUC wants to buy IIUC-TV subscriptions. We know that 6 people can share a single IIUC-TV subscription, IIUC. Also, the cost of one IIUC-TV subscription is X Taka. Determine the minimum total cost that the group of N friends will incur to use IIUC-TV, IIUC

Input-Output Screenshot:

```
1 package javatvcost;
2 import java.util.Scanner;
3
4 public class JavaTvCost {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int N = sc.nextInt();
9         int X = sc.nextInt();
10
11         int subs = (N + 5) / 6;
12         int total = subs * X;
13
14         System.out.println(total);
15     }
16 }
17
18
19
```

 Output - JavaTvCost (run) x

 run:
 10 8000
16000

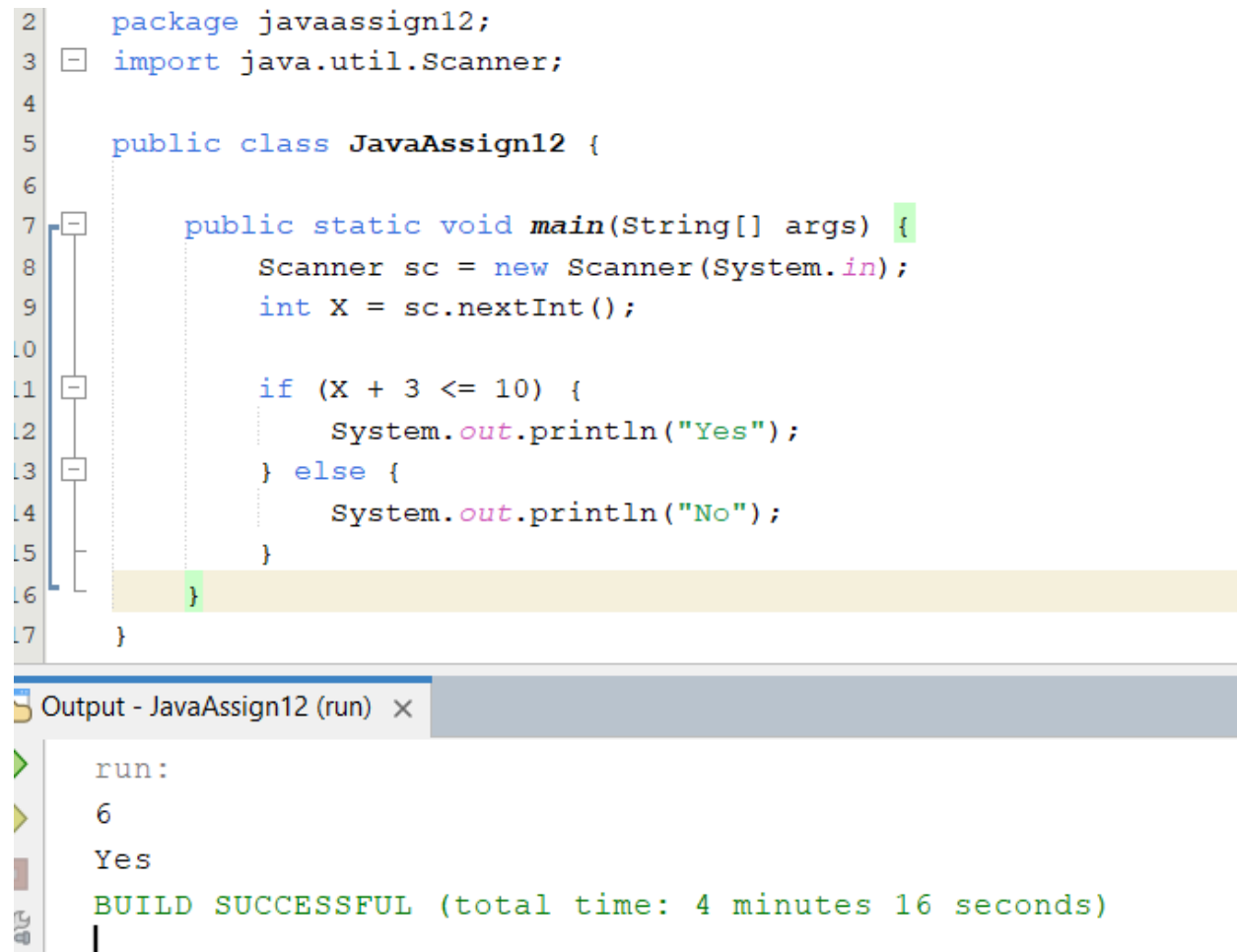
 BUILD SUCCESSFUL (total time: 11 seconds)

Approach:

Here I took the number of friends N and cost per subscription X as input. Since one subscription can serve 6 people I calculated the required number using $(N + 5) / 6$. Then I multiplied it with X to get the total minimum cost and printed it.

Problem 7: A student has to submit 3 assignments before 10 pm, and he starts to do the assignments at X pm. Each assignment takes him 1 hour to complete. Can you tell whether he'll be able to complete all assignments on time?

Input-Output Screenshot:



```
2 package javaassign12;
3 import java.util.Scanner;
4
5 public class JavaAssign12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int X = sc.nextInt();
10
11         if (X + 3 <= 10) {
12             System.out.println("Yes");
13         } else {
14             System.out.println("No");
15         }
16     }
17 }
```

Output - JavaAssign12 (run) x

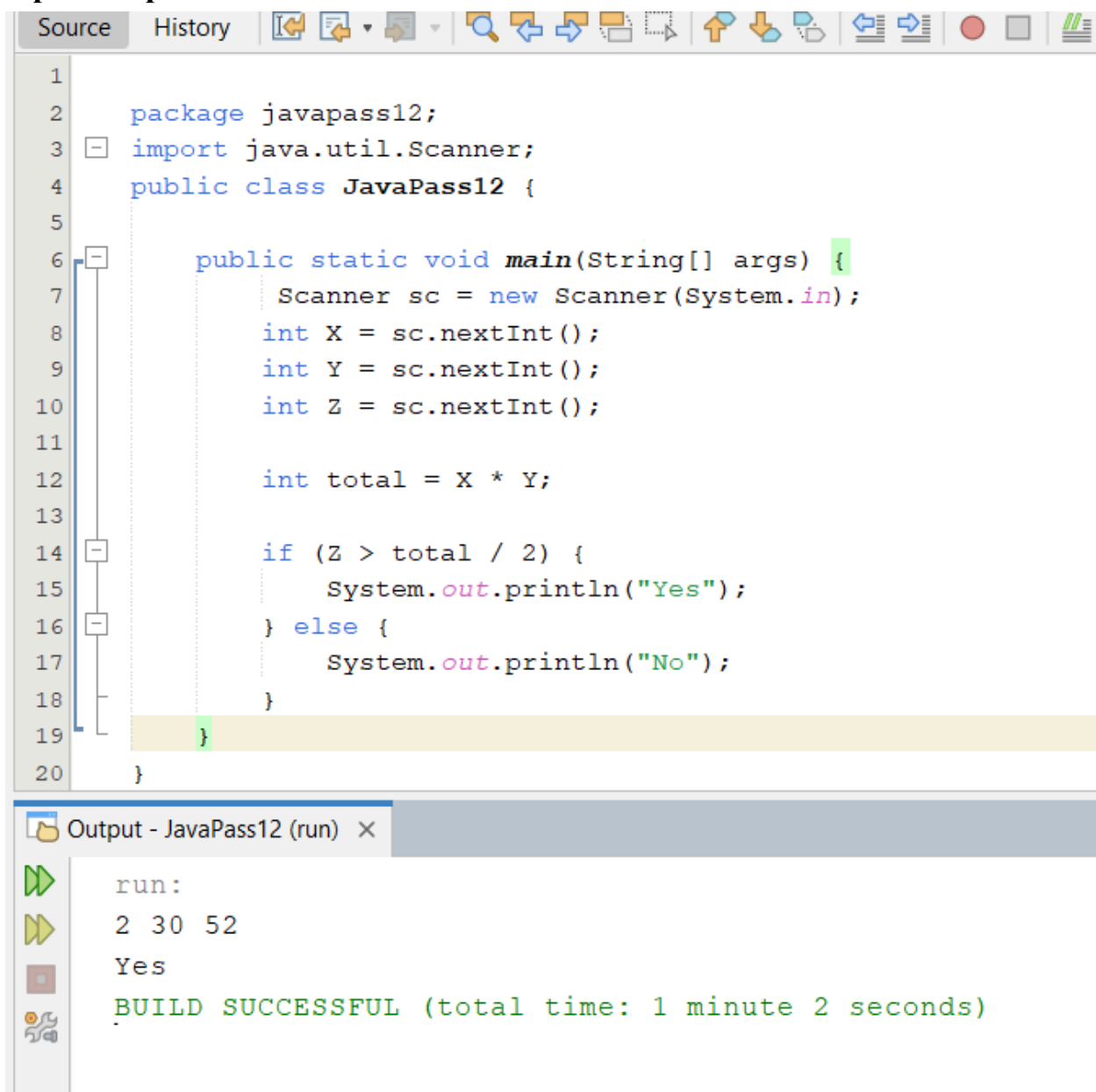
```
run:
6
Yes
BUILD SUCCESSFUL (total time: 4 minutes 16 seconds)
```

Approach:

In this program I took the starting time X as input. Since each assignment takes 1 hour I added 3 hours to X. Then I checked whether the total time is less than or equal to 10 pm and printed Yes or No accordingly.

Problem 8: In IIUC, there are X sections, and each section has Y students. The semester-end results are in, and a total of Z students passed the exams. Assuming that all students appeared for the exams, find whether the number of students who passed was strictly greater than 50%.

Input-Output Screenshot:



The screenshot displays an IDE with a Java source file and its execution output. The source code defines a class `JavaPass12` with a `main` method that reads three integers `X`, `Y`, and `Z` from the user. It calculates the total number of students as `X * Y` and checks if `Z` is greater than half of the total. The output window shows the program was run successfully with inputs `2 30 52`, resulting in the output `Yes`.

```
1 package javapass12;
2
3 import java.util.Scanner;
4 public class JavaPass12 {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int X = sc.nextInt();
9         int Y = sc.nextInt();
10        int Z = sc.nextInt();
11
12        int total = X * Y;
13
14        if (Z > total / 2) {
15            System.out.println("Yes");
16        } else {
17            System.out.println("No");
18        }
19    }
20 }
```

Output - JavaPass12 (run) x

```
run:
2 30 52
Yes
BUILD SUCCESSFUL (total time: 1 minute 2 seconds)
```

Approach:

Here I took X sections Y students per section and Z passed students as input. I calculated total students using $X * Y$. Then I checked whether Z is strictly greater than half of total students and printed the result.

Problem 9: Recently You joined a new company. At this company, employees must work X hours each day from Saturday to Wednesday. Also, in this company, Wednesday is called Chill Day; employees only have to work for Y hours ($Y < X$) on Wednesday. Determine the total number of working hours in one week.

Input-Output Screenshot:

```
2 package javaworkhour12;
3 import java.util.Scanner;
4
5 public class JavaWorkHour12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int X = sc.nextInt();
10        int Y = sc.nextInt();
11
12        int total = 4 * X + Y;
13
14        System.out.println(total);
15    }
16 }
17
```

Output - JavaWorkHour12 (run) ×

```
run:
10 3
43
BUILD SUCCESSFUL (total time: 51 seconds)
```

Approach:

In this program I took X as working hours for normal days and Y for Wednesday. Since there are 4 full working days and one reduced day I calculated total hours as $4 * X + Y$ and printed it.

Problem 10: An IIUCian has three water bottles. At any point, if at least two of them are empty, he will fill them up. But if at most one bottle is empty, he will wait and not fill them up now. You are given three integers: B1, B2, and B3. If B1=1, then the first bottle is full. If B1=0, then the first bottle is empty. Similarly, B2 denotes whether the second bottle is full or empty, and B3 denotes it for the third bottle. Output "Water filling time", if he has to fill the bottles now. If not, output "Not now".

Input-Output Screenshot:

```
import java.util.Scanner;

public class JavaBottle12 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int B1 = sc.nextInt();
        int B2 = sc.nextInt();
        int B3 = sc.nextInt();

        int empty = 0;

        if (B1 == 0) empty++;
        if (B2 == 0) empty++;
        if (B3 == 0) empty++;

        if (empty >= 2) {
            System.out.println("Water filling time");
        } else {
            System.out.println("Not now");
        }
    }
}
```

Output - JavaBottle12 (run) ×

```
run:
1 3 4
Not now
BUILD SUCCESSFUL (total time: 52 seconds)
```

Approach:

Here I took three inputs B1,B2 and B3 where 0 means empty and 1 means full. I counted how many bottles are empty. If at least two bottles are empty I printed Water filling time otherwise I printed Not now.

Problem 11: You know that 1 kg of pulp can be used to make 1000 pages and 1 notebook consists of 100 pages. Suppose a notebook factory receives N kg of pulp. How many notebooks can be made from that?

Input-Output Screenshot:

```
2 package javanote12;
3 import java.util.Scanner;
4
5 public class JavaNote12 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int N = sc.nextInt();
10        int pages = N * 1000;
11        int notebooks = pages / 100;
12
13        System.out.println(notebooks);
14    }
15 }
```

Output - JavaNote12 (run) ×

run:
10000
100000
BUILD SUCCESSFUL (total time: 6 seconds)

Approach:

In this program I took the amount of pulp N in kg as input. Since 1 kg produces 1000 pages I calculated total pages as $N * 1000$. Then I divided by 100 pages per notebook to find total notebooks and printed the result.

Problem 12: There are N children, and an IIUCian wants to give each of them 1 candy. He already has X candies. To buy the rest, he visits a candy shop. In the shop, packets containing exactly 4 candies are available. Determine the minimum number of candy packets he must buy so that he can give 1 candy packet to each of the N children

Input-Output Screenshot:

```
2 package javacandy12;
3 import java.util.Scanner;
4
5 public class JavaCandy12{
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int N = sc.nextInt();
9         int X = sc.nextInt();
10
11         int need = N - X;
12
13         if (need <= 0) {
14             System.out.println(0);
15         } else {
16             int packets = (need + 3) / 4;
17             System.out.println(packets);
18         }
19     }
20 }
```

Output - JavaCandy12 (run) x

run:
5 3
1
BUILD SUCCESSFUL (total time: 3 seconds)

Approach:

Here I took number of children N and available candies X as input. I calculated required candies as $N - X$. If it is less than or equal to zero I printed 0 otherwise I used $(\text{need} + 3) / 4$ to calculate minimum packets and printed it.

Problem 13: An IIUCian has reached the finals of the Annual Inter-university Declamation contest. For the finals, students were asked to prepare 10 topics. However, he was only able to prepare three topics, numbered A, B, and C; he is totally blank about the other topics. This means he can only win the contest if he gets to speak on topics A, B, or C. On the contest day, he gets topic X. Determine whether he has any chances of winning the competition. Print "Yes" if it is possible to win the contest, else print "No".

Input-Output Screenshot:

```
2 package javacontest12;
3 import java.util.Scanner;
4 public class JavaContest12 {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int A = sc.nextInt();
9         int B = sc.nextInt();
10        int C = sc.nextInt();
11        int X = sc.nextInt();
12
13        if (X == A || X == B || X == C) {
14            System.out.println("Yes");
15        } else {
16            System.out.println("No");
17        }
18    }
19 }
```

Output - JavaContest12 (run) ×

run:
1 2 3 2
Yes
BUILD SUCCESSFUL (total time: 5 minutes 9 seconds)

Approach:

In this program I took three prepared topics A,B and C and the given topic X as input. I checked whether X is equal to A or B or C using logical OR. If yes I printed Yes otherwise No.

Problem 14: There are 4 companies in the markets of Chittagong, A, B, C, and D. This year, a. Company A made a profit of P lakh, b. Company B made a profit of Q lakh, c. Company C made a profit of R lakh, d. Company D made a profit of S lakh. There is said to be a monopoly in the market if the profit made by one company is strictly greater than the sum of profits made by all other companies. Determine whether there is a monopoly in the market.

Input-Output Screenshot:

```
2 package javacheck12;
3 import java.util.Scanner;
4
5 public class JavaCheck12 {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         int P = sc.nextInt();
9         int Q = sc.nextInt();
10        int R = sc.nextInt();
11        int S = sc.nextInt();
12
13        if (P > Q + R + S || Q > P + R + S ||
14            R > P + Q + S || S > P + Q + R) {
15            System.out.println("Yes");
16        } else {
17            System.out.println("No");
18        }
19    }
20 }
```

Output - JavaCheck12 (run) x

run:
3 5 4 7
No
BUILD SUCCESSFUL (total time: 1 minute 33 seconds)

Approach:

Here I took the profits of four companies P Q R and S as input. I checked whether any one company's profit is strictly greater than the sum of the other three. If such a condition is true then there is a monopoly and I printed "Yes". Otherwise I printed "No".

Problem 15: There are three people sitting in a room: Dean Sir, Chairman Sir, and Coordinator Sir. They need to decide on the temperature to set on the air conditioner. Everyone has a demand each: a. Alice wants the temperature to be at least A degrees. b. Bob wants the temperature to be at most B degrees. c. Charlie wants the temperature to be at least C degrees. Can they all agree on some temperature, or not?

Input-Output Screenshot:

```
package javatemp12;
import java.util.Scanner;

public class JavaTemp12 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int A = sc.nextInt(); // min
        int B = sc.nextInt(); // max
        int C = sc.nextInt(); // min

        int lower = Math.max(A, C);

        if (lower <= B) {
            System.out.println("Yes");
        } else {
            System.out.println("No");
        }
    }
}
```

Output - JavaTemp12 (run) ×

```
run:
18 24 19
Yes
BUILD SUCCESSFUL (total time: 19 seconds)
```

Approach:

In this program I took three values A B and C as input representing minimum and maximum temperature conditions. I found the maximum of A and C since both require minimum temperatures. Then I checked whether this value is less than or equal to B (maximum allowed). If yes all conditions are satisfied so I printed "Yes" otherwise "No".